

Introduction of Inheritance

Assisted Problems

1. Animal Hierarchy

- **Description:** Create a hierarchy where `Animal` is the superclass, and `Dog`, `Cat`, and `Bird` are subclasses. Each subclass has a unique behavior.
- **Tasks:**
 - Define a superclass `Animal` with attributes `name` and `age`, and a method `makeSound()`.
 - Define subclasses `Dog`, `Cat`, and `Bird`, each with a unique implementation of `makeSound()`.
- **Goal:** Learn basic inheritance, method overriding, and polymorphism with simple classes.

```
package assistedproblems.animalhierarchy;

public class Animal {

    //fields
    public String name;
    public int age;

    //constructor to initialize values
    public Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }

    //method to make sound
```

```
public void makeSound(String sound) {
    System.out.println("Animal sound...." + sound);
}
//method to display details
public void display() {
    System.out.println("Name : " + this.name);
    System.out.println("Age : " + this.age);
}
}
```

```
package assistedproblems.animalhierarchy;

public class Dog extends Animal {

    //constructor to initialize values
    public Dog(String name, int age) {
        super(name, age);
    }

    //overriding makeSound() method
    @Override
    public void makeSound(String sound) {
        System.out.println("Dog Sound..." + sound);
    }
}
```

```
package assistedproblems.animalhierarchy;

public class Cat extends Animal {

    //constructor to initialize values
    public Cat(String name, int age) {
        super(name, age);
    }

    //overriding makeSound() method
    @Override
    public void makeSound(String sound) {
        System.out.println("Cat Sound..." + sound);
    }
}
```

```
package assistedproblems.animalhierarchy;

public class Bird extends Animal {

    //constructor to initialize values
    public Bird(String name, int age) {
        super(name, age);
    }

    //overriding makeSound() method
    @Override
    public void makeSound(String sound) {
```

```
        System.out.println("bird Sound..." + sound);  
    }  
}
```

```
package assistedproblems.animalhierarchy;  
  
import java.util.Scanner;  
public class Controller {  
    public static void main(String[] args) {  
        //creating Scanner class object  
        Scanner input = new Scanner(System.in);  
  
        //taking user input for animal  
        System.out.print("Enter animal's name : ");  
        String name = input.nextLine();  
        System.out.print("Enter animal's age : ");  
        int age = input.nextInt();  
        input.nextLine();  
        System.out.print("Enter animal's sound : ");  
        String sound = input.nextLine();  
  
        //creating Animal class object  
        Animal animal = new Animal(name, age);  
  
        //printing details of animal  
        System.out.println("\nDetails of animal .....");  
        animal.display();  
    }  
}
```

```
//calling makeSound() method
animal.makeSound(sound);

//taking user input for dog
System.out.print("\nEnter dog's name : ");
name = input.nextLine();

System.out.print("Enter dog's age : ");
age = input.nextInt();
input.nextLine();
System.out.print("Enter dog's sound : ");
sound = input.nextLine();

//creating Dog class object
Dog dog = new Dog(name, age);

//printing details of dog
System.out.println("\nDetails of Dog .....");
dog.display();
//calling makeSound() method
dog.makeSound(sound);

//taking user input for Cat
System.out.print("\nEnter Cat's name : ");
name = input.nextLine();
System.out.print("Enter Cat's age : ");
age = input.nextInt();
input.nextLine();
System.out.print("Enter Cat's sound : ");
sound = input.nextLine();
```

```
//creating Cat class object
Cat cat = new Cat(name, age);

//printing details of Cat
System.out.println("\nDetails of Cat .....");
cat.display();
//calling makeSound() method
cat.makeSound(sound);

//taking user input for bird
System.out.print("\nEnter bird's name : ");
name = input.nextLine();
System.out.print("Enter bird's age : ");
age = input.nextInt();
input.nextLine();
System.out.print("Enter bird's sound : ");
sound = input.nextLine();

//creating bird class object
Bird bird = new Bird(name, age);

//printing details of bird
System.out.println("\nDetails of bird .....");
bird.display();
//calling makeSound() method
bird.makeSound(sound);

//closing input object
input.close();
```

}

}

2. Employee Management System

- **Description:** Create an **Employee** hierarchy for different employee types such as **Manager**, **Developer**, and **Intern**.
- **Tasks:**
 - Define a base class **Employee** with attributes like **name**, **id**, and **salary**, and a method **displayDetails()**.
 - Define subclasses **Manager**, **Developer**, and **Intern** with unique attributes for each, like **teamSize** for **Manager** and **programmingLanguage** for **Developer**.
- **Goal:** Practice inheritance by creating subclasses with specific attributes and overriding superclass methods.

```
package assistedproblems.employeeManagement;

public class Employee {
    //fields
    public String name;
    public String id;
    public double salary;

    //constructor to initialize values
    public Employee(String name, String id, double
salary) {
        this.name = name;
        this.id = id;
        this.salary = salary;
    }
}
```

```
//method to display details
public void display() {
    System.out.println("Name : " + this.name);
    System.out.println("Id : " + this.id);
    System.out.println("Salary : " + this.salary);
}
}
```

```
package assistedproblems.employeemanagement;

public class Manager extends Employee {
    //fields
    public int teamSize;

    //constructor to initialize values
    public Manager(String name, String id, double
salary, int teamSize) {
        super(name, id, salary);
        this.teamSize = teamSize;
    }
    //overriding display() method
    @Override
    public void display() {
        super.display();
        System.out.println("Team size : " +
this.teamSize);
    }
}
```

```
package assistedproblems.employeemanagement;

public class Developer extends Employee {
    //fields
    public String programmingLanguage;

    //constructor to initialize values
    public Developer(String name, String id, double salary, String programmingLanguage) {
        super(name, id, salary);
        this.programmingLanguage = programmingLanguage;
    }
    //overriding display() method
    @Override
    public void display() {
        super.display();
        System.out.println("Programming language : " +
this.programmingLanguage);
    }
}
```

```
package assistedproblems.employeemanagement;

public class Intern extends Employee {
    //constructor to initialize values
    public Intern(String name, String id, double salary)
{
```

```
        super(name, id, salary);  
    }  
}
```

```
package assistedproblems.employeemanagement;  
  
import java.util.Scanner;  
  
public class Controller {  
    public static void main(String[] args) {  
        //creating Scanner class object  
        Scanner input = new Scanner(System.in);  
  
        //taking user input for Employee  
        System.out.print("\nEnter employee's name : ");  
        String name = input.nextLine();  
        System.out.print("Enter employee's ID : ");  
        String id = input.nextLine();  
        System.out.print("Enter employee's salary : ");  
        double salary = input.nextDouble();  
  
        //creating Employee class object  
        Employee employee = new Employee(name, id,  
salary);  
  
        //printing details of employee  
        System.out.println("\nDetails of Employee  
.....");  
        employee.display();  
    }  
}
```

```
input.nextLine();
//taking user input for manager
System.out.print("\nEnter manager's name : ");
name = input.nextLine();
System.out.print("Enter manager's ID : ");
id = input.nextLine();
System.out.print("Enter manager's salary: ");
salary = input.nextDouble();
System.out.print("Enter manager's team size :
");
int teamSize = input.nextInt();

//creating manager class object
Manager manager = new Manager(name, id, salary,
teamSize);

//printing details of manager
System.out.println("\nDetails of manager
.....");
manager.display();

input.nextLine();
//taking user input for developer
System.out.print("\nEnter developer's name : ");
name = input.nextLine();
System.out.print("Enter developer's ID : ");
id = input.nextLine();
System.out.print("Enter developer's salary : ");
salary = input.nextDouble();
```

```
input.nextLine();
System.out.print("Enter programming language :
");
String programmingLanguage = input.nextLine();

//creating developer class object
Developer developer = new Developer(name, id,
salary, programmingLanguage);

//printing details of developer
System.out.println("\nDetails of developer
.....");
developer.display();

//taking user input for intern
System.out.print("\nEnter intern's name : ");
name = input.nextLine();
System.out.print("Enter intern's ID : ");
id = input.nextLine();
System.out.print("Enter intern's salary : ");
salary = input.nextDouble();

//creating intern class object
Intern intern = new Intern(name, id, salary);

//printing details of intern
System.out.println("\nDetails of intern .....");
intern.display();
//closing input object
input.close();
```

```
}  
}
```

3. Vehicle and Transport System

- **Description:** Design a vehicle hierarchy where **Vehicle** is the superclass, and **Car**, **Truck**, and **Motorcycle** are subclasses with unique attributes.
- **Tasks:**
 - Define a superclass **Vehicle** with **maxSpeed** and **fuelType** attributes and a method **displayInfo()**.
 - Define subclasses **Car**, **Truck**, and **Motorcycle**, each with additional attributes, such as **seatCapacity** for **Car**.
 - Demonstrate polymorphism by storing objects of different subclasses in an array of **Vehicle** type and calling **displayInfo()** on each.
- **Goal:** Understand how inheritance helps in organizing shared and unique features across subclasses and use polymorphism for dynamic method calls.

```
package assistedproblems.vehicleandtransportsystem;  
  
public class Vehicle {  
    public double maxSpeed;  
    public String fuelType;  
  
    //constructor to initialize values  
    public Vehicle(double maxSpeed, String fuelType) {  
        this.maxSpeed = maxSpeed;  
        this.fuelType = fuelType;  
    }  
  
    //method to display details  
    public void displayInfo() {  
        System.out.println("Max Speed : " +  
this.maxSpeed);  
    }  
}
```

```
        System.out.println("Fuel type : " +  
this.fuelType);  
    }  
}
```

```
package assistedproblems.vehicleandtransportsystem;  
  
public class Car extends Vehicle{  
    //fields  
    public int seatCapacity;  
  
    //constructor to initialize values  
    public Car(double maxSpeed, String fuelType, int  
seatCapacity) {  
        super(maxSpeed, fuelType);  
        this.seatCapacity = seatCapacity;  
    }  
    //overriding display method  
    @Override  
    public void displayInfo() {  
        super.displayInfo();  
        System.out.println("seat capacity : " +  
this.seatCapacity);  
        System.out.println();  
    }  
}
```

```
package assistedproblems.vehicleandtransportsystem;

public class Truck extends Vehicle {

    //constructor to initialize values
    public Truck(double maxSpeed, String fuelType) {
        super(maxSpeed, fuelType);
    }
}
```

```
package assistedproblems.vehicleandtransportsystem;

public class Motorcycle extends Vehicle {

    //constructor to initialize values
    public Motorcycle(double maxSpeed, String fuelType)
    {
        super(maxSpeed, fuelType);
    }
}
```

```
package assistedproblems.vehicleandtransportsystem;
import java.util.Scanner;

public class Controller {
    public static void main(String[] args) {
```

```
//creating Scanner class object
Scanner input = new Scanner(System.in);

//taking user input for Vehicle
System.out.print("Enter vehicle's max speed :
");
double maxSpeed = input.nextDouble();
input.nextLine();
System.out.print("Enter vehicle's fuel type :
");
String fuelType = input.nextLine();

//creating Vehicle class object
Vehicle vehicle = new Vehicle(maxSpeed,
fuelType);

//taking user input for car
System.out.print("Enter car's max speed : ");
maxSpeed = input.nextDouble();
input.nextLine();
System.out.print("Enter car's fuel type : ");
fuelType = input.nextLine();
System.out.print("Enter seat capacity : ");
int seatCapacity = input.nextInt();
input.nextLine();

//creating car class object
Car car = new Car(maxSpeed, fuelType,
seatCapacity);
```

```
//taking user input for truck
System.out.print("Enter truck's max speed : ");
maxSpeed = input.nextDouble();
input.nextLine();
System.out.print("Enter truck's fuel type : ");
fuelType = input.nextLine();

//creating truck class object
Truck truck = new Truck(maxSpeed, fuelType);

//taking user input for motorcycle
System.out.print("Enter motorcycle's max speed : ");
maxSpeed = input.nextDouble();
input.nextLine();
System.out.print("Enter motorcycle's fuel type : ");
fuelType = input.nextLine();

//creating motorcycle class object
Motorcycle motorcycle = new Motorcycle(maxSpeed, fuelType);

//creating array of vehicle type
Vehicle[] vehicles = {vehicle, car, truck, motorcycle};

//printing details of vehicle
System.out.println("\nDetails of vehicle
.....");
```

```
vehicles[0].displayInfo();

//printing details of car
System.out.println("\nDetails of car .....");
vehicles[1].displayInfo();

//printing details of truck
System.out.println("\nDetails of truck .....");
vehicles[2].displayInfo();

//printing details of motorcycle
System.out.println("\nDetails of motorcycle
.....");
vehicles[3].displayInfo();

//closing input object
input.close();
}
}
```

Single Inheritance

Sample Problem 1: Library Management with Books and Authors

- **Description:** Model a **Book** system where **Book** is the superclass, and **Author** is a subclass.
- **Tasks:**
 - Define a superclass **Book** with attributes like **title** and **publicationYear**.
 - Define a subclass **Author** with additional attributes like **name** and **bio**.
 - Create a method **displayInfo()** to show details of the book and its author.

- **Goal:** Practice single inheritance by extending the base class and adding more specific details in the subclass.

```
package assistedproblems.LibraryManagement;

public class Book {
    //fields
    public String title;
    public int publicationYear;

    //constructor to initialize values
    public Book(String title, int publicationYear) {
        this.title = title;
        this.publicationYear = publicationYear;
    }

    //method to display information
    public void display() {
        System.out.println("Title : " + this.title);
        System.out.println("Publication year : " +
this.publicationYear);
    }
}
```

```
package assistedproblems.LibraryManagement;

public class Author extends Book {
    //fields
```

```
public String authorName;
public String bio;

//constructor to initialize values
public Author(String title, int publicationYear,
String authorName, String bio) {

    //calling super class constructor
    super(title, publicationYear);

    this.authorName = authorName;
    this.bio = bio;
}

//overriding display method
@Override
public void display() {
    super.display();
    System.out.println("Author name : " +
this.authorName);
    System.out.println("Bio : " + this.bio);
}
}
```

```
package assistedproblems.LibraryManagement;
import java.util.Scanner;

public class Controller {
    public static void main(String[] args) {
```

```
//creating Scanner class object
Scanner input = new Scanner(System.in);

//taking input for BookDetails
System.out.print("Enter Book's title : ");
String title = input.nextLine();
System.out.print("Enter Book's publication year
: ");
int publicationYear = input.nextInt();

input.nextLine();
//taking input for author
System.out.print("Enter Author's name : ");
String authorName = input.nextLine();
System.out.print("Enter Bio : ");
String bio = input.nextLine();

//creating Author class object
Author author = new Author(title,
publicationYear, authorName, bio);

//printing details
System.out.println();
author.display();
}
}
```

Sample Problem 2: Smart Home Devices

- **Description:** Create a hierarchy for a smart home system where `Device` is the superclass and `Thermostat` is a subclass.
- **Tasks:**
 - Define a superclass `Device` with attributes like `deviceId` and `status`.
 - Create a subclass `Thermostat` with additional attributes like `temperatureSetting`.
 - Implement a method `displayStatus()` to show each device's current settings.
- **Goal:** Understand single inheritance by adding specific attributes to a subclass, keeping the superclass general.

```
package singleinheritance.smarthomedevices;

public class Device {
    //fields
    public String deviceId;
    public String status;

    //constructor to initialize values
    public Device(String deviceId, String status) {
        this.deviceId = deviceId;
        this.status = status;
    }

    //method to print status
    public void displayStatus() {
        System.out.println("Device id : " +
this.deviceId);
        System.out.println("Status : " + this.status);
    }
}
```

```
package singleinheritance.smarthomedevices;

public class Thermostat extends Device {
    //fields
    public String temperatureSetting;

    //constructor to initialize values
    public Thermostat(String deviceId, String status,
String temperatureSetting) {

        //calling super class constructor
        super(deviceId, status);

        this.temperatureSetting = temperatureSetting;
    }
    //overriding the display method
    @Override
    public void displayStatus() {
        super.displayStatus();
        System.out.println("Temperature setting : " +
this.temperatureSetting);
    }
}
```

```
package singleinheritance.smarthomedevices;

import java.util.Scanner;

public class Controller {
```

```
public static void main(String[] args) {

    //creating Scanner class object
    Scanner input = new Scanner(System.in);

    //taking input for Thermostat class object
    System.out.println("\nFetching details for
thermostat object ....");
    System.out.print("Enter device ID : ");
    String deviceId = input.nextLine();
    System.out.print("Enter status : ");
    String status = input.nextLine();

    //taking input for temperatureSetting
    System.out.print("Enter Temperature setting :
");
    String temperatureSetting = input.nextLine();

    //creating Thermostat class object
    Thermostat thermostat = new Thermostat(deviceId,
status, temperatureSetting);

    //printing details
    System.out.println();
    System.out.println("\nDetails of thermostat
object.....");
    thermostat.displayStatus();

    //taking input for Device class object
    System.out.println("\nFetching details for
```

```
Device object ....");
    System.out.print("Enter device ID : ");
    deviceId = input.nextLine();
    System.out.print("Enter status : ");
    status = input.nextLine();

    //creating Device class object
    Device device = new Device(deviceId, status);

    //printing details
    System.out.println();
    System.out.println("\nDetails of Device
object.....");
    device.displayStatus();

    //closing the input
    input.close();
}
}
```

Multilevel Inheritance

Sample Problem 1: Online Retail Order Management

- **Description:** Create a multilevel hierarchy to manage orders, where **Order** is the base class, **ShippedOrder** is a subclass, and **DeliveredOrder** extends **ShippedOrder**.
- **Tasks:**
 - Define a base class **Order** with common attributes like **orderId** and **orderDate**.
 - Create a subclass **ShippedOrder** with additional attributes like **trackingNumber**.

- Create another subclass `DeliveredOrder` extending `ShippedOrder`, adding a `deliveryDate` attribute.
- Implement a method `getOrderStatus()` to return the current order status based on the class level.
- **Goal:** Explore multilevel inheritance, showing how attributes and methods can be added across a chain of classes.

```
package
multilevelinheritance.onlinetailordermanagement;

public class Order {
    //fields
    public String orderId;
    public String orderDate;

    //constructor to initialize values
    public Order(String orderId, String orderDate) {
        this.orderId = orderId;
        this.orderDate = orderDate;
    }

    //method to get order status
    public String getOrderStatus() {
        return "Order Status : Order placed ";
    }

    //method to display details
    public void display() {
        System.out.println("Order Id : " +
this.orderId);
        System.out.println("Order date : " +
this.orderDate);
    }
}
```

```
package
multilevelinheritance.onlinetailordermanagement;

public class ShippedOrder extends Order {
    //fields
    public int trackingNumber;

    //constructor to initialize values
    public ShippedOrder(String orderId, String
orderIdDate, int trackingNumber) {
        //calling super class constructor
        super(orderId, orderIdDate);

        this.trackingNumber = trackingNumber;
    }
    //overriding method getOrderStatus()
    @Override
    public String getOrderStatus() {
        return "Order Status : Order shipped ";
    }
    //overriding method display()
    @Override
    public void display() {

        //calling super class display method
        super.display();
        System.out.println("Tracking number : " +
```

```
this.trackingNumber);  
    }  
}
```

```
package  
multilevelinheritance.onlinetailordermanagement;  
  
public class DeliveredOrder extends ShippedOrder {  
    //fields  
    public String deliveryDate;  
  
    //constructor to initialize values  
    public DeliveredOrder(String orderId, String  
orderDate, int trackingNumber, String deliveryDate) {  
        //calling super class constructor  
        super(orderId, orderDate, trackingNumber);  
        this.deliveryDate = deliveryDate;  
    }  
    //overriding method getOrderStatus()  
    @Override  
    public String getOrderStatus() {  
        return "Order Status : Order delivered ";  
    }  
    //overriding method display()  
    @Override  
    public void display() {  
  
        //calling super class display method  
        super.display();  
        System.out.println("Delivery date : " +
```

```
this.deliveryDate);  
    }  
}
```

```
package  
multilevelinheritance.onlinetailordermanagement;  
  
import java.util.Scanner;  
  
public class Controller {  
    public static void main(String[] args) {  
  
        //creating Scanner class object  
        Scanner input = new Scanner(System.in);  
  
        //taking input for Order  
        System.out.println("\nFetching details for  
placed order....");  
        System.out.print("Enter order ID : ");  
        String orderId = input.nextLine();  
        System.out.print("Enter order date : ");  
        String orderDate = input.nextLine();  
  
        //creating Order class object  
        Order order = new Order(orderId, orderDate);  
  
        //printing details  
        System.out.println();  
        System.out.println("\nDetails of Placed
```

```
order.....");
    order.display();

    //calling method getOrderStatus()
    System.out.println(order.getOrderStatus());

    //taking input for shipped Order
    System.out.println("\nFetching details for a
shipped order....");
    System.out.print("Enter order ID : ");
    orderId = input.nextLine();
    System.out.print("Enter order date : ");
    orderDate = input.nextLine();
    System.out.print("Enter tracking number
(Integer) : ");
    int trackingNumber = input.nextInt();
    input.nextLine();

    //creating ShippedOrder class object
    ShippedOrder shippedOrder = new
ShippedOrder(orderId, orderDate, trackingNumber);

    //printing details
    System.out.println();
    System.out.println("\nDetails of Shipped
order.....");
    shippedOrder.display();

    //calling method getOrderStatus()
```

```
System.out.println(shippedOrder.getOrderStatus());

    //taking input for delivered Order
    System.out.println("\nFetching details for a
delivered order....");
    System.out.print("Enter order ID : ");
    orderId = input.nextLine();
    System.out.print("Enter order date : ");
    orderDate = input.nextLine();
    System.out.print("Enter tracking number
(Integer) : ");
    trackingNumber = input.nextInt();
    input.nextLine();
    System.out.print("Enter delivery date : ");
    String deliveryDate = input.nextLine();

    //creating DeliveredOrder class object
    DeliveredOrder deliveredOrder = new
DeliveredOrder(orderId, orderDate, trackingNumber,
deliveryDate);

    //printing details
    System.out.println();
    System.out.println("\nDetails of Delivered
order.....");
    deliveredOrder.display();

    //calling method getOrderStatus()

System.out.println(deliveredOrder.getOrderStatus());
```

```
//closing the input
input.close();
}
}
```

Sample Problem 2: Educational Course Hierarchy

- **Description:** Model a course system where **Course** is the base class, **OnlineCourse** is a subclass, and **PaidOnlineCourse** extends **OnlineCourse**.
- **Tasks:**
 - Define a superclass **Course** with attributes like **courseName** and **duration**.
 - Define **OnlineCourse** to add attributes such as **platform** and **isRecorded**.
 - Define **PaidOnlineCourse** to add **fee** and **discount**.
- **Goal:** Demonstrate how each level of inheritance builds on the previous, adding complexity to the system.

```
package
multilevelinheritance.EducationalCourseHierarchy;

public class Course {
    //fields
    public String courseName;
    public int duration;

    //constructor to initialize values
    public Course(String courseName, int duration) {
        this.courseName = courseName;
        this.duration = duration;
    }
}
```

```
}  
//method to print details  
public void display() {  
    System.out.println("Course name : " +  
this.courseName);  
    System.out.println("Duration : " + this.duration  
+ " months");  
}  
}
```

```
package  
multilevelinheritance.EducationalCourseHierarchy;  
  
public class OnlineCourse extends Course {  
    //fields  
    public String platform;  
    public boolean isRecorded;  
  
    //constructor to initialize values  
    public OnlineCourse(String courseName, int duration,  
String platform, Boolean isRecorded) {  
        super(courseName, duration);  
        this.platform = platform;  
        this.isRecorded = isRecorded;  
    }  
    //overriding method to print details  
    @Override  
    public void display() {  
        super.display();  
    }  
}
```

```
        System.out.println("Platform : " +
this.platform);
        System.out.println("Is recorded ? " +
this.isRecorded);
    }
}
```

```
package
multilevelinheritance.EducationalCourseHierarchy;

public class PaidOnlineCourse extends OnlineCourse {
    //fields
    public double fee;
    public double discount;

    //constructor to initialize values
    public PaidOnlineCourse(String courseName, int
duration, String platform, boolean isRecorded, double
fee, double discount) {
        //calling super class constructor
        super(courseName, duration, platform,
isRecorded);
        this.fee = fee;
        this.discount = discount;
    }
    //overriding method to print details
    @Override
    public void display() {
        super.display();
    }
}
```

```
        System.out.println("Fee : " + this.fee);
        System.out.println("Discount : " +
this.discount);
    }
}
```

```
package
multilevelinheritance.EducationalCourseHierarchy;

import java.util.Scanner;

public class Controller {
    public static void main(String[] args) {
        //creating scanner class object
        Scanner input = new Scanner(System.in);

        //taking input for course details
        System.out.println("\nFetching details for
course....");
        System.out.print("Enter courseName : ");
        String courseName = input.nextLine();
        System.out.print("Enter duration (in months) :
");
        int duration = input.nextInt();

        //creating Course class object
        Course course = new Course(courseName,
duration);
    }
}
```

```
//printing details
System.out.println();
System.out.println("Details of course.....");
course.display();

input.nextLine();
//taking input for online course details
System.out.println("\nFetching details for
online course....");
System.out.print("Enter courseName : ");
courseName = input.nextLine();
System.out.print("Enter duration (in months) :
");
duration = input.nextInt();
input.nextLine();
System.out.print("Enter platform : ");
String platform = input.nextLine();

//taking input if the session is recorded or not
System.out.print("Enter course is recorded or
not (true/false) : ");
boolean isRecorded = input.nextBoolean();
input.nextLine();

//creating OnlineCourse class object
OnlineCourse onlineCourse = new
OnlineCourse(courseName, duration, platform,
isRecorded);

//printing details
```

```
System.out.println();
System.out.println("Details of Online
course.....");
onlineCourse.display();

//taking input for paid online course details
System.out.println("\nFetching details for paid
online course....");
System.out.print("Enter courseName : ");
courseName = input.nextLine();
System.out.print("Enter duration (in months) :
");
duration = input.nextInt();
input.nextLine();
System.out.print("Enter platform : ");
platform = input.nextLine();

//taking input if the session is recorded or not
System.out.print("Enter course is recorded or
not (true/false) : ");
isRecorded = input.nextBoolean();
input.nextLine();
//taking input for additional details
System.out.print("Enter fee : ");
double fee = input.nextDouble();
System.out.print("Enter discount : ");
double discount = input.nextDouble();

//creating PaidOnlineCourse class object
PaidOnlineCourse paidOnlineCourse = new
```

```
PaidOnlineCourse(courseName, duration, platform,
isRecorded, fee, discount);

    //printing details
    System.out.println();
    System.out.println("Details of Online
course.....");
    onlineCourse.display();

    //closing the input
    input.close();
}
}
```

Hierarchical Inheritance

Sample Problem 1: Bank Account Types

- **Description:** Model a banking system with different account types using hierarchical inheritance. `BankAccount` is the superclass, with `SavingsAccount`, `CheckingAccount`, and `FixedDepositAccount` as subclasses.
- **Tasks:**
 - Define a base class `BankAccount` with attributes like `accountNumber` and `balance`.
 - Define subclasses `SavingsAccount`, `CheckingAccount`, and `FixedDepositAccount`, each with unique attributes like `interestRate` for `SavingsAccount` and `withdrawalLimit` for `CheckingAccount`.
 - Implement a method `displayAccountType()` in each subclass to specify the account type.
- **Goal:** Explore hierarchical inheritance, demonstrating how each subclass can have unique attributes while inheriting from a shared superclass.

```
package hierarchicalinheritance.bankaccounttypes;

public class BankAccount {
    //fields
    public String accountHolder;
    public int accountNumber;
    public double balance;

    //constructor to initialize values
    public BankAccount(String accountHolder, int
accountNumber, double balance) {
        this.accountHolder = accountHolder;
        this.accountNumber = accountNumber;
        this.balance = balance;
    }
    //method to display details
    public void display() {
        System.out.println("Account Holder : " +
this.accountHolder);
        System.out.println("Account Number : " +
this.accountNumber);
        System.out.println("Balance : " + this.balance);
    }
}
```

```
package hierarchicalinheritance.bankaccounttypes;
```

```
public class SavingAccount extends BankAccount {
    //fields
    public double interestRate;

    //constructor to initialize values
    public SavingAccount(String accountHolder, int
accountNumber, double balance, double interestRate) {
        super(accountHolder, accountNumber, balance);
        this.interestRate = interestRate;
    }
    //overriding method display()
    @Override
    public void display() {
        super.display();
        System.out.println("Interest rate : " +
this.interestRate);
    }
}
```

```
package hierarchicalinheritance.bankaccounttypes;

public class CheckingAccount extends BankAccount {
    //fields
    public double withdrawLimit;

    //constructor to initialize values
    public CheckingAccount(String accountHolder, int
accountNumber, double balance, double withdrawLimit) {
        super(accountHolder, accountNumber, balance);
    }
}
```

```
        this.withdrawLimit = withdrawLimit;
    }
    //overriding method display()
    @Override
    public void display() {
        super.display();
        System.out.println("Withdraw limit : " +
this.withdrawLimit);
    }
}
```

```
package hierarchicalinheritance.bankaccounttypes;

public class FixedDepositAccount extends BankAccount {
    //fields
    public double FDamount;

    //constructor to initialize values
    public FixedDepositAccount(String accountHolder, int
accountNumber, double balance, double FDamount) {
        super(accountHolder, accountNumber, balance);
        this.FDamount = FDamount;
    }
    //overriding method display()
    @Override
    public void display() {
        super.display();
        System.out.println("FD amount : " +
this.FDamount);
    }
}
```

```
}  
}
```

```
package hierarchicalinheritance.bankaccounttypes;  
  
import java.util.Scanner;  
  
public class Controller {  
    public static void main(String[] args) {  
        //creating Scanner class object  
        Scanner input = new Scanner(System.in);  
  
        //taking user input for Bank account  
        System.out.println("\nFetching normal bank  
account details...");  
        System.out.print("Enter account holder's name :  
");  
        String accountHolder = input.nextLine();  
        System.out.print("Enter account number : ");  
        int accountNumber = input.nextInt();  
        input.nextLine();  
        System.out.print("Enter balance : ");  
        double balance = input.nextDouble();  
        input.nextLine();  
  
        //creating BankAccount class object  
        BankAccount bankAccount = new  
BankAccount(accountHolder, accountNumber, balance);
```

```
//printing details of bank account
System.out.println("\nDetails of normal bank
account.....");
bankAccount.display();

//taking user input for saving Bank account
System.out.println("\nFetching saving bank
account details...");
System.out.print("Enter account holder's name :
");
accountHolder = input.nextLine();
System.out.print("Enter account number : ");
accountNumber = input.nextInt();
input.nextLine();
System.out.print("Enter balance : ");
balance = input.nextDouble();
input.nextLine();
System.out.print("Enter interest rate : ");
double interestRate = input.nextDouble();
input.nextLine();

//creating SavingAccount class object
SavingAccount savingAccount = new
SavingAccount(accountHolder, accountNumber, balance,
interestRate);

//printing details of saving bank account
System.out.println("\nDetails of saving bank
account.....");
savingAccount.display();
```

```
//taking user input for Checking account
System.out.println("\nFetching checking bank
account details...");
System.out.print("Enter account holder's name :
");

accountHolder = input.nextLine();
System.out.print("Enter account number : ");
accountNumber = input.nextInt();
input.nextLine();
System.out.print("Enter balance : ");
balance = input.nextDouble();
input.nextLine();
System.out.print("Enter withdraw limit : ");
double withdrawLimit = input.nextDouble();
input.nextLine();

//creating CheckingAccount class object
CheckingAccount checkingAccount = new
CheckingAccount(accountHolder, accountNumber, balance,
withdrawLimit);

//printing details of checking account
System.out.println("\nDetails of checking
account.....");
checkingAccount.display();

//taking user input for fixed deposit account
System.out.println("\nFetching fixed deposit
account details...");
```

```
System.out.print("Enter account holder's name :
");
accountHolder = input.nextLine();
System.out.print("Enter account number : ");
accountNumber = input.nextInt();
input.nextLine();
System.out.print("Enter balance : ");
balance = input.nextDouble();
input.nextLine();
System.out.print("Enter fixed deposit amount :
");
double FDamount = input.nextDouble();
input.nextLine();

//creating FixedDepositAccount class object
FixedDepositAccount fixedDepositAccount = new
FixedDepositAccount(accountHolder, accountNumber,
balance, withdrawLimit);

//printing details of fixed deposit account
System.out.println("\nDetails of fixed deposit
account.....");
fixedDepositAccount.display();

//closing input object
input.close();
}
}
```

Sample Problem 2: School System with Different Roles

- **Description:** Create a hierarchy for a school system where **Person** is the superclass, and **Teacher**, **Student**, and **Staff** are subclasses.
- **Tasks:**
 - Define a superclass **Person** with common attributes like **name** and **age**.
 - Define subclasses **Teacher**, **Student**, and **Staff** with specific attributes (e.g., **subject** for **Teacher** and **grade** for **Student**).
 - Each subclass should have a method like **displayRole()** that describes the role.
- **Goal:** Demonstrate hierarchical inheritance by modeling different roles in a school, each with shared and unique characteristics.

```
package hierarchicalinheritance.schoolsystem;

public class Person {
    //fields
    public String name;
    public int age;

    //constructor to initialize values
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    //method to display role
    public void displayRole() {
        System.out.println("Role : Person");
    }
    //method to print details
    public void display() {
        System.out.println("Name : " + this.name);
    }
}
```

```
        System.out.println("Age : " + this.age);  
    }  
}
```

```
package hierarchicalinheritance.schoolsystem;  
  
public class Teacher extends Person{  
    //fields  
    public String subject;  
  
    //constructor to initialize values  
    public Teacher(String name, int age, String subject)  
    {  
        super(name, age);  
        this.subject = subject;  
    }  
    //overriding method displayRole()  
    @Override  
    public void displayRole() {  
        System.out.println("Role : Teacher");  
    }  
    //overriding method display()  
    @Override  
    public void display() {  
        super.display();  
        System.out.println("Subject : " + this.subject);  
    }  
}
```

```
package hierarchicalinheritance.schoolsystem;

public class Student extends Person {
    //fields
    public String grade;

    //constructor to initialize values
    public Student(String name, int age, String grade) {
        super(name, age);
        this.grade = grade;
    }
    //overriding method displayRole()
    @Override
    public void displayRole() {
        System.out.println("Role : Student");
    }
    //overriding method display()
    @Override
    public void display() {
        super.display();
        System.out.println("Grade : " + this.grade);
    }
}
```

```
package hierarchicalinheritance.schoolsystem;

public class Staff extends Person {
    //fields
    public String designation;
```

```
//constructor to initialize values
public Staff(String name, int age, String
designation) {
    super(name, age);
    this.designation = designation;
}
//overriding method displayRole()
@Override
public void displayRole() {
    System.out.println("Role : Staff");
}
//overriding method display()
@Override
public void display() {
    super.display();
    System.out.println("Designation : " +
this.designation);
}
}
```

```
package hierarchicalinheritance.schoolsystem;
import java.util.Scanner;

public class Controller {
    public static void main(String[] args) {
        //creating Scanner class object
        Scanner input = new Scanner(System.in);
    }
}
```

```
//taking user input for Person
System.out.println("\nFetching person's
details...");
System.out.print("Enter name : ");
String name = input.nextLine();
System.out.print("Enter age : ");
int age = input.nextInt();
input.nextLine();

//creating Person class object
Person person = new Person(name, age);

//printing details of person
System.out.println("\nDetails of Person.....");
person.display();
//calling displayRole() method
person.displayRole();

//taking user input for Teacher
System.out.println("\nFetching teacher's
details...");
System.out.print("Enter name : ");
name = input.nextLine();
System.out.print("Enter age : ");
age = input.nextInt();
input.nextLine();
System.out.print("Enter Subject : ");
String subject = input.nextLine();

//creating Teacher class object
```

```
Teacher teacher = new Teacher(name, age,
subject);

//printing details of teacher
System.out.println("\nDetails of Teacher.....");
teacher.display();
//calling displayRole() method
teacher.displayRole();

//taking user input for student
System.out.println("\nFetching student's
details...");
System.out.print("Enter name : ");
name = input.nextLine();
System.out.print("Enter age : ");
age = input.nextInt();
input.nextLine();
System.out.print("Enter Grade : ");
String grade = input.nextLine();

//creating Student class object
Student student = new Student(name, age,
subject);

//printing details of student
System.out.println("\nDetails of student.....");
student.display();
//calling displayRole() method
student.displayRole();
```

```
//taking user input for staff
System.out.println("\nFetching staff's
details...");
System.out.print("Enter name : ");
name = input.nextLine();
System.out.print("Enter age : ");
age = input.nextInt();
input.nextLine();
System.out.print("Enter designation : ");
String designation = input.nextLine();

//creating staff class object
Staff staff = new Staff(name, age, designation);

//printing details of staff
System.out.println("\nDetails of staff.....");
staff.display();
//calling displayRole() method
staff.displayRole();

//closing input object
input.close();
}
}
```

Hybrid Inheritance (Simulating Multiple Inheritance)

Since Java doesn't support multiple inheritance directly, hybrid inheritance is typically achieved through **interfaces**.

Sample Problem 1: Restaurant Management System with Hybrid Inheritance

- **Description:** Model a restaurant system where **Person** is the superclass and **Chef** and **Waiter** are subclasses. Both **Chef** and **Waiter** should implement a **Worker** interface that requires a **performDuties()** method.
- **Tasks:**
 - Define a superclass **Person** with attributes like **name** and **id**.
 - Create an interface **Worker** with a method **performDuties()**.
 - Define subclasses **Chef** and **Waiter** that inherit from **Person** and implement the **Worker** interface, each providing a unique implementation of **performDuties()**.
- **Goal:** Practice hybrid inheritance by combining inheritance and interfaces, giving multiple behaviors to the same objects.

```
package hybridinheritance.restaurantmanagement;

public class Person {
    //fields
    public String name;
    public String id;

    //constructor to initialize values
    public Person(String name, String id) {
        this.name = name;
        this.id = id;
    }
    //method to display details
    public void display() {
        System.out.println("Name : " + this.name);
        System.out.println("Id : " + this.id);
    }
}
```

```
package hybridinheritance.reataurantmanagement;

public interface Worker {
    //abstract method to be implemented in chef and
    waiter class
    public void performDuties();
}
```

```
package hybridinheritance.reataurantmanagement;

public class Chef extends Person implements Worker {
    //fields
    public String speciality;

    //constructor to initialize values
    public Chef(String name, String id, String
speciality) {
        super(name, id);
        this.speciality = speciality;
    }
    //implementation of performDuties() method
    @Override
    public void performDuties() {
        System.out.println("Prepares delicious " +
this.speciality);
    }
}
```

```
package hybridinheritance.reataurantmanagement;

public class Waiter extends Person implements Worker {
    //constructor to initialize values
    public Waiter(String name, String id) {
        super(name, id);
    }
    //implementation of performDuties() method
    @Override
    public void performDuties() {
        System.out.println("Serves dishes..");
    }
}
```

```
package hybridinheritance.reataurantmanagement;
import java.util.Scanner;

public class controller {
    public static void main(String[] args) {
        //creating Scanner class object
        Scanner input = new Scanner(System.in);

        //taking user input for Person
        System.out.println("\nFetching person's
details...");
        System.out.print("Enter name : ");
        String name = input.nextLine();
        System.out.print("Enter id : ");
    }
}
```

```
String id = input.nextLine();

//creating Person class object
Person person = new Person(name, id);

//printing details of person
System.out.println("\nDetails of Person.....");
person.display();

//taking user input for chef
System.out.println("\nFetching chef's
details...");
System.out.print("Enter name : ");
name = input.nextLine();
System.out.print("Enter id : ");
id = input.nextLine();
System.out.print("Enter speciality : ");
String speciality = input.nextLine();

//creating Chef class object
Chef chef = new Chef(name, id, speciality);

//printing details of chef
System.out.println("\nDetails of chef.....");
chef.display();
//calling method performDuties()
chef.performDuties();

//taking user input for waiter
System.out.println("\nFetching waiter's
```

```
details...");
    System.out.print("Enter name : ");
    name = input.nextLine();
    System.out.print("Enter id : ");
    id = input.nextLine();

    //creating Waiter class object
    Waiter waiter = new Waiter(name, id);

    //printing details of waiter
    System.out.println("\nDetails of waiter.....");
    waiter.display();
    //calling method performDuties()
    waiter.performDuties();

    //closing input
    input.close();
}
}
```

Sample Problem 2: Vehicle Management System with Hybrid Inheritance

- **Description:** Model a vehicle system where `Vehicle` is the superclass and `ElectricVehicle` and `PetrolVehicle` are subclasses. Additionally, create a `Refuelable` interface implemented by `PetrolVehicle`.
- **Tasks:**
 - Define a superclass `Vehicle` with attributes like `maxSpeed` and `model`.
 - Create an interface `Refuelable` with a method `refuel()`.

- Define subclasses `ElectricVehicle` and `PetrolVehicle`.
`PetrolVehicle` should implement `Refuelable`, while `ElectricVehicle` include a `charge()` method.
- **Goal:** Use hybrid inheritance by having `PetrolVehicle` implement both `Vehicle` and `Refuelable`, demonstrating how Java interfaces allow adding multiple behaviors.

```
package hybridinheritance.vehicleManagementsystem;

public class Vehicle {
    //fields
    public double maxSpeed;
    public String model;

    //constructor to initialize values
    public Vehicle(double maxSpeed, String model) {
        this.maxSpeed = maxSpeed;
        this.model = model;
    }
    //method to print details
    public void display() {
        System.out.println("Max speed : " +
this.maxSpeed);
        System.out.println("Model : " + this.model);
    }
}
```

```
package hybridinheritance.vehicleManagementsystem;

public interface Refuelable {

    //method to refuel
    public void refuel();
}
```

```
package hybridinheritance.vehicleManagementsystem;

public class ElectricVehicle extends Vehicle implements
Refuelable {

    //constructor to initialize values
    public ElectricVehicle(double maxSpeed, String
model) {
        super(maxSpeed, model);
    }
    //implementation of refuel() method
    public void refuel() {
        System.out.println("Refuelable.....");
    }
}
```

```
package hybridinheritance.vehicleManagementsystem;

public class PetrolVehicle extends Vehicle implements
Refuelable {
    //constructor to initialize values
    public PetrolVehicle(double maxSpeed, String model)
    {
        super(maxSpeed, model);
    }
    //implementation of refuel() method
    public void refuel() {
        System.out.println("Refuelable.....");
    }
}
```

```
package hybridinheritance.vehicleManagementsystem;
import java.util.Scanner;

public class Controller {
    public static void main(String[] args) {
        //creating Scanner class object
        Scanner input = new Scanner(System.in);

        //taking user input for Vehicle
        System.out.println("\nFetching vehicle's
details...");
        System.out.print("Enter max speed : ");
        double maxSpeed = input.nextInt();
        input.nextLine();
    }
}
```

```
System.out.print("Enter model : ");
String model = input.nextLine();

//creating Vehicle class object
Vehicle vehicle = new Vehicle(maxSpeed, model);

//printing details of vehicle
System.out.println("\nDetails of vehicle.....");
vehicle.display();

//taking user input for Electric Vehicle
System.out.println("\nFetching Electric
Vehicle's details...");
System.out.print("Enter max speed : ");
maxSpeed = input.nextInt();
input.nextLine();
System.out.print("Enter model : ");
model = input.nextLine();

//creating ElectricVehicle class object
ElectricVehicle electricVehicle = new
ElectricVehicle(maxSpeed, model);

//printing details of Electric Vehicle
System.out.println("\nDetails of Electric
Vehicle.....");
electricVehicle.display();
//calling method performDuties()
electricVehicle.refuel();
```

```
//taking user input for petrol Vehicle
System.out.println("\nFetching petrol Vehicle's
details...");
System.out.print("Enter max speed : ");
maxSpeed = input.nextInt();
input.nextLine();
System.out.print("Enter model : ");
model = input.nextLine();

//creating petrol Vehicle class object
PetrolVehicle petrolVehicle = new
PetrolVehicle(maxSpeed, model);

//printing details of petrol Vehicle
System.out.println("\nDetails of petrol
Vehicle.....");
petrolVehicle.display();
//calling method performDuties()
petrolVehicle.refuel();

//closing the input
input.close();
}
}
```

1. Favor Composition Over Inheritance

- Use composition instead
- instead of inheritance when a class can be described as "has-a" rather than "is-a".
- This avoids tight coupling and provides greater flexibility.

2. Ensure Proper Use of **is-a** Relationship

- Use inheritance only when the subclass truly extends the behavior of the superclass, maintaining the "is-a" relationship.
 - Avoid misusing inheritance for code reuse.
-

3. Follow Liskov Substitution Principle

- Subclasses should be substitutable for their superclasses without breaking the application.
 - Ensure overridden methods maintain the expected behavior of the superclass.
-

4. Avoid Deep Inheritance Hierarchies

- Keep the inheritance hierarchy shallow to reduce complexity and improve maintainability.
 - Deep hierarchies can make debugging and understanding the code difficult.
-

5. Mark Superclass Methods **final** If Needed

- Prevent subclasses from overriding critical methods by marking them **final**.
 - This ensures essential functionality remains unchanged.
-

6. Use **@Override** Annotation

- Always use **@Override** to explicitly indicate that a method is being overridden.
 - This helps catch errors during compilation if the method signature is incorrect.
-

7. Minimize Public Fields in Superclasses

- Use private or protected fields with proper getters and setters.
- This prevents unintended access or modification by subclasses.

8. Avoid Overloading Alongside Overriding

- Overloading methods with similar names and parameters in subclasses can lead to confusion.
 - Ensure clarity by distinctly separating overridden methods from overloaded ones.
-

9. Prefer Abstract Classes for Partial Implementation

- Use abstract classes to define a blueprint with partial implementation for related classes.
 - Abstract classes provide flexibility while enforcing a consistent structure.
-

10. Use Interfaces for Multiple Inheritance

- Java does not support multiple inheritance through classes. Use interfaces to achieve multiple inheritance-like behavior.
 - This helps avoid the "diamond problem."
-

11. Document Inheritance Behavior

- Clearly document the purpose and expected behavior of the superclass and its methods.
 - Provide details on how subclasses should override or extend the methods.
-

12. Avoid Overriding Methods Unnecessarily

- Override methods only when necessary and when the subclass needs to modify or extend the behavior of the superclass.
-

13. Be Cautious with Constructors

- Call the superclass constructor explicitly in the subclass constructor using `super()`.
- Avoid calling non-final methods from constructors to prevent issues with uninitialized state in subclasses.

14. Use Polymorphism Effectively

- Design systems to leverage polymorphism where superclass references are used to interact with subclass objects.
 - This promotes flexibility and extensibility.
-

15. Beware of Fragile Base Class Problem

- Changes to the superclass can inadvertently affect all subclasses.
 - Minimize dependencies and changes to the superclass once it is widely used.
-

16. Test Subclass and Superclass Interactions

- Thoroughly test how the subclass interacts with inherited methods and state.
 - Ensure changes in the subclass do not break the expected behavior of the superclass.
-

17. Avoid Inheriting from Concrete Classes

- Prefer inheriting from abstract classes or interfaces rather than concrete classes.
 - This avoids tight coupling to a specific implementation.
-

18. Consider Using Delegation for Special Cases

- When specific behavior is needed in some instances but not others, delegation may be a better choice than inheritance.
- This promotes better separation of concerns.